

Reviewer Pack — Projective Plane Incidence Matrix Deterministic Communicatio...

Entry 710cd3eca0ca · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 03:03:40 UTC

Projective Plane Incidence Matrix Deterministic Communication Complexity

Entry ID: 710cd3eca0ca **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 03:03:40 UTC

1. Conjecture

Field A (mathematical branch): Finite Geometry **Field B** (complexity object): Deterministic Communication Complexity

Statement:

For a function f whose communication matrix M_f is the incidence matrix of a projective plane $PG(2, q)$, the deterministic communication complexity $D(f)$ is $\Theta(q^2)$. This holds for all q where $PG(2, q)$ exists (q a prime power).

Rationale (proposer's reasoning):

The incidence matrix of a projective plane imposes a highly symmetric, structured correlation between inputs. This regularity forces parties to exchange $\Omega(q^2)$ bits to resolve ambiguities in the matrix's entries, as the dual structure of points and lines creates a combinatorial barrier to efficient coordination.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c9ba813bbc79bc55

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def is_prime(q):
    if q <= 1:
        return False
    for i in range(2, int(q**0.5) + 1):
        if q % i == 0:
            return False
    return True

def generate_projective_plane(q):
    if not is_prime(q):
        raise ValueError("q must be a prime power")

    points = list(range(q * (q + 1)))
    lines = []

    for i in range(q):
        line = [i]
        for j in range(1, q + 1):
            line.append((i * j) % q)
        lines.append(line)

    for i in range(q + 1):
        line = [(q * (q + 1) - i)]
        for j in range(1, q + 1):
            line.append(((q * (q + 1) - i) * j) % q)
        lines.append(line)

    return points, lines

def incidence_matrix(points, lines):
    n = len(points)
    M = [[0] * n for _ in range(n)]

    for line in lines:
        for point in line:
            M[point][line.index(point)] = 1
```

```

    return M

def matrix_multiplication(A, B):
    m = len(A)
    p = len(B[0])
    q = len(B)
    C = [[0] * p for _ in range(m)]

    for i in range(m):
        for j in range(p):
            for k in range(q):
                C[i][j] += A[i][k] * B[k][j]

    return C

def gaussian_elimination(A, b):
    n = len(A)
    Augmented = [A[i] + [b[i]] for i in range(n)]

    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(Augmented[j][i]) > abs(Augmented[max_row][i]):
                max_row = j

        Augmented[i], Augmented[max_row] = Augmented[max_row], Augmented[i]

        factor = Augmented[i][i]
        for j in range(i, n + 1):
            Augmented[i][j] /= factor

        for j in range(n):
            if j != i:
                factor = Augmented[j][i]
                for k in range(i, n + 1):
                    Augmented[j][k] -= factor * Augmented[i][k]

    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = Augmented[i][-1]
        for j in range(i + 1, n):
            x[i] -= Augmented[i][j] * x[j]

    return x

def discrepancy_method(M):
    m, n = len(M), len(M[0])
    A = [[0] * (n - 1) for _ in range(m)]
    b = [0] * m

    for i in range(m):
        for j in range(n - 1):
            A[i][j] = M[i][j]
        b[i] = sum(M[i])

    x = gaussian_elimination(A, b)

```

```

discrepancy = max(abs(x[i]) for i in range(n - 1))

return discrepancy

def run_trial(seed: int) -> dict:
    random.seed(seed)
    q_values = [2, 3, 4]
    results = []

    for q in q_values:
        try:
            points, lines = generate_projective_plane(q)
            M_f = incidence_matrix(points, lines)

            D_f = discrepancy_method(M_f)
            metric_value = D_f
            conjecture_holds = abs(D_f - q**2) < 1e-6
            counterexample = "" if conjecture_holds else f"q={q}, D(f)={D_f}"
        except Exception as e:
            metric_value = None
            conjecture_holds = False
            counterexample = str(e)

        results.append({
            "metric_name": "discrepancy",
            "metric_value": metric_value,
            "instances_tested": 1,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        })

    return {
        "seed": seed,
        "trials": results
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30))

    all_results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        all_results.extend(result["trials"])

    mean_value = sum(r["metric_value"] for r in all_results if r["metric_value"] is not None) / len(all_results)
    std_value = (sum((r["metric_value"] - mean_value)**2 for r in all_results if r["metric_value"] is not None) / len(all_results))**0.5
    support_fraction = sum(1 for r in all_results if r["conjecture_holds"]) / len(all_results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in all_results):
        counterexample = next(r["counterexample"] for r in all_results if not r["conjecture_holds"])
        first_failing_seed = next(r["seed"] for r in all_results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ut of range'}, {'metric_name': 'discrepancy', 'metric_value': None, 'instances_tested': 1, 'conjecture': '2^p-1 is prime'},
TRIAL: {'seed': 167, 'trials': [{'metric_name': 'discrepancy', 'metric_value': None, 'instances_tested': 1, 'conjecture': '2^p-1 is prime'},
TRIAL: {'seed': 191, 'trials': [{'metric_name': 'discrepancy', 'metric_value': None, 'instances_tested': 1, 'conjecture': '2^p-1 is prime'},
TRIAL: {'seed': 211, 'trials': [{'metric_name': 'discrepancy', 'metric_value': None, 'instances_tested': 1, 'conjecture': '2^p-1 is prime'},
TRIAL: {'seed': 233, 'trials': [{'metric_name': 'discrepancy', 'metric_value': None, 'instances_tested': 1, 'conjecture': '2^p-1 is prime'}]}

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing reliable data; multiple counterexamples suggest potential issues but insufficient evidence due to test failure | next: Implement robust error handling for prime power validation and re-run tests with $q=2,3,4$ (smallest prime powers) to verify conjecture

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	95950
2	preregistration	ollama_remote	qwen3:8b	0	0	19978
3	novelty	ollama_remote	qwen3:8b	0	0	16558
4	novelty	ollama_remote	qwen3:8b	0	0	13062
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14983
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14625
7	judge	ollama_remote	qwen3:8b	0	0	15502

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 190657 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/710cd3eca0ca.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/710cd3eca0ca.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/710cd3eca0ca.tar.gz` (if generated)